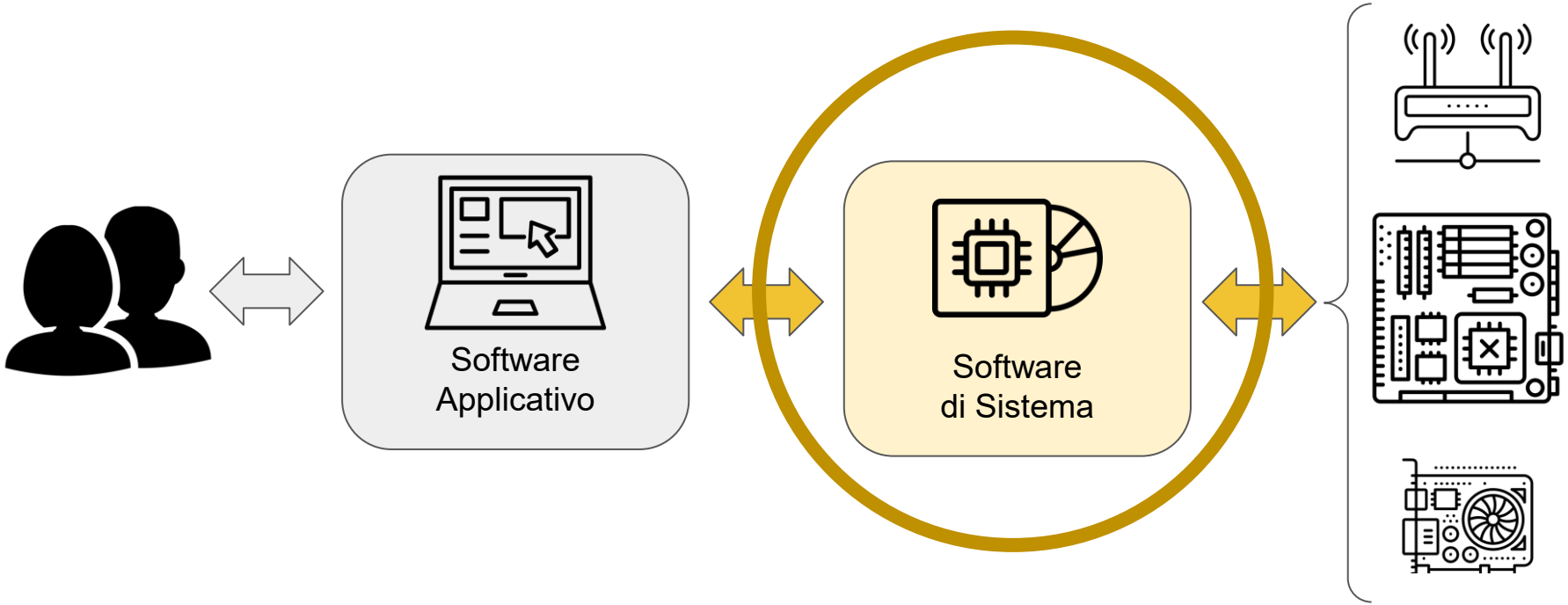
Introduzione

2025-26

© G. Malnati, 2021-24

Cos'è la programmazione di sistema?



Cos'è la programmazione di sistema?

- La programmazione di sistema fornisce le fondamenta che supportano la programmazione applicativa
  - Creando il software necessario a permettere ai tanti componenti dell'ecosistema di elaborazione (CPU, memoria, periferiche, rete, …) di operare come un organismo integrato
- La programmazione di sistema è orientata ai contesti dove le risorse sono limitate
  - Richiede la capacità di organizzare il codice tenendo conto che ciascun byte e ciascun ciclo di CPU è rilevante
- La quantità di codice di sistema coinvolto nel permettere ad una semplice applicazione di essere eseguita è sorprendente
  - Sistema operativo, driver delle periferiche, compilatore, linker, loader, librerie di vario tipo…

© G. Malnati, 2021-24

Ambiti della programmazione di sistema

Stack di rete
Sistemi operativi
Elaborazione dei media
Simulazioni scientifiche Renderizzazione del testo
Dispositivi embedded
Gestione della memoria
Crittografia
Linguaggi ad alto livello
Giochi
File System
Macchine virtuali
Device Drivers
DBMS

Linguaggi per la programmazione di sistema

- Scrivere codice ad alte prestazioni richiede l'uso di un linguaggio che permetta al programmatore di controllare come siano usate le risorse di memoria, processore, I/O, …
  - Questo esclude, a priori, tutti quei linguaggi che **adottano un sistema di gestione automatico della memoria** [*garbage collector*] (come Java, JavaScript, Python, C#, Go, …)
  - Riducendo le opzioni possibili sostanzialmente all'uso di C, C++ e Rust (oltre a Swift, in ambiente Apple)

- C e C++: offrono ottimi risultati in termini di prestazioni, ma sono problematici per problemi intrinseci legati all'uso incorretto dei puntatori che portano a situazioni critiche chiamate *undefined behavior*
- Rust combina tale livello di controllo con un sistema di tipi alquanto sofisticato, in grado di identificare, in fase di compilazione, un'ampia gamma di errori, dall'uso incorretto dei puntatori alle interferenze tra thread
  - Garantendo che se un programma riesce ad essere compilato, non contiene comportamenti non definiti (che non vuol dire che funzionerà certamente, ma che avrà un comportamento riproducibile)
  - *By design non permette l'undefined behavior*!

RUST

- Il compilatore ha un modello molto più sofisticato della gestione della memoria e l'idea di possesso della memoria da parte di un puntatore
- Permette di gestire in modo efficace la programmazione concorrente grazie ad un modello astratto della programmazione concorrente che impedisce di scrivere operazioni che causano malfunzionamenti.

- Dunque, se un programma è compilato NON contiene comportamenti non definiti.
- Questo non vuol dire che il programma è *giusto* (se scrivo un ciclo infinito, il programma girerà all'infinito….)

Abilità richieste

- Coding
  - Algoritmi (inserimento, ricerca, ordinamento, …)
  - Strutture dati (stringhe, liste, insiemi, mappe, …)
  - Lettura e scrittura di file (conversioni di formato, segmentazione del testo, codifica di dati binari, …)
  - Conoscenza approfondita del linguaggio C
- Architetture di elaborazione e elettronica digitale
  - CPU, memoria, bus di comunicazione, periferiche, interrupt, DMA …
  - Nozioni di base sui sistemi operativi
- Capacità di analisi di un problema e pensiero critico
  - Definizione e comprensione dei requisiti
  - Schematizzazione tramite diagrammi (entità/relazioni, sequenze di eventi, gerarchie di vario tipo, …)
- Attitudine alla risoluzione dei problemi
  - Elaborazione e raffinamento dei propri modelli mentali

Abilità da sviluppare

- Motivazione
  - Il corso è impegnativo per la vastità dei concetti che esplora così come per la loro intrinseca complessità
  - In base alle inclinazioni e sensibilità personali, può anche apparire arido
  - Può essere opportuno mettere a fuoco molto bene le motivazioni che hanno portato a scegliere il percorso magistrale di Ingegneria Informatica per trovare le energie necessarie ad impegnarsi nello studio, dando ad esso un senso nella prospettiva del proprio progetto di vita professionale
- Organizzazione e gestione del tempo
  - La presenza di due percorsi paralleli richiede un'attenta pianificazione dei propri tempi di studio, per non lasciare indietro qualche concetto che poi diventa molto difficile recuperare
  - Parallelamente, occorre trovare il giusto ritmo, sapendo concedersi il riposo necessario, per evitare di saturare le proprie capacità mentali

© G. Malnati, 2021-24

API Programming - Obiettivi attesi

- Conoscenza del linguaggio Rust e suo utilizzo per la realizzazione di applicativi di media complessità
- Comprensione approfondita dei meccanismi soggiacenti l'esecuzione di un programma nel contesto di un sistema operativo
- Comprensione delle problematiche introdotte dalla programmazione concorrente e capacità di realizzare algoritmi in grado di operare correttamente in contesti multi-thread e asincroni
- (per chi svolge il progetto) Capacità di lavorare in gruppo, organizzando il lavoro così da coinvolgere tutti e provando a gestire le problematiche connesse, sia sul piano tecnico che su quello umano

Questionario di autovalutazione iniziale

- Preparato dal prof. Malnati
- Da consegnarsi entro 24 / 02

https://forms.gle/JHLZG5Q3E715nLzH8

https://qrco.de/bgdkCQ

Gruppo Discord

- Per rimanere connessi:

  https://discord.gg/6cfYQxnl